

CS 6650 — Midterm Mastery

Steps I – III: Sharing, Reflections & Crash Recovery

Eroniction Presley

Northeastern University — Khoury College of Computer Sciences

March 2, 2026

Outline

- 1 Step I: Homework 6 Results
- 2 Step II: What I Found Valuable
- 3 Step III: Crash & Recovery Experiment
- 4 Key Takeaways

Step I: Homework 6 — Product API at Scale

- Deployed **Go Product API** (Gin framework) to AWS ECS/Fargate via **Terraform**
- Load tested with **Locust** — scaled to **4,000 RPS with zero failures**
- CloudWatch metrics: CPU scaled linearly with load, memory flat at **7.67%**
- Compared **HttpUser vs FastHttpUser** and **1 vs 4 Locust workers**

Key Insight: When CPU rises linearly but memory is flat and there's no algorithmic headroom — the answer is *more compute, not better code*.

Step I: Scaling Observations & Peer Comparison

My Results

- Product search bounded at 100 products/request
- CPU bottleneck, not memory
- FastHttpUser: higher throughput, lower latency
- 4 workers $\hat{=}$ 1 worker, but not 4x (Amdahl's Law)

Piazza Peer Discussion

- Peers saw different throughput on similar configs
- Factors: search term distribution, instance warm-up, Locust client CPU limits
- Shared on “SharingHomework6” thread

“From Make It Work to Make It Scale”

My mental model for scaling decisions:

- ① **Measure** — identify the bottleneck with metrics
- ② **Can you optimize?** — is there algorithmic headroom?
- ③ **Scale direction** — CPU-bound & stateless? Scale out. Stateful? Extract state first.
- ④ **Design for failure** — health checks, auto-replacement, redundancy from day one

Scale Up = bigger hardware (simple but has a ceiling, single point of failure)

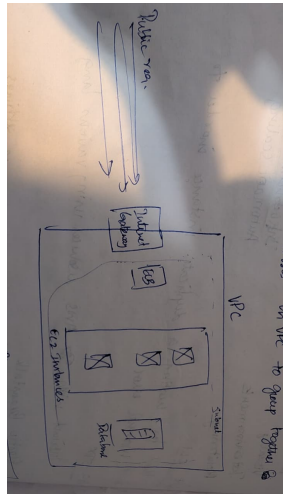
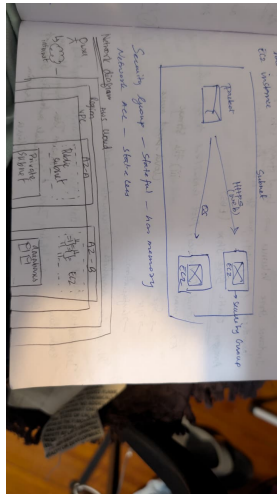
Scale Out = more instances (complex but fault tolerant, theoretically unlimited)

Step II: AWS Architecture — My Mental Model

Key Components

- EC2 instances in subnets
- Security Groups (stateful) vs Network ACLs (stateless)
- Target Groups + ALB for routing
- Request flow: Internet Gateway → ELB → EC2 → DB
- All within a VPC

*Assignment 5 (Terraform) assembled these pieces;
Assignment 6 pushed them to scale.*



Step II: Concurrency — Root of All Distributed Problems

Assignment 3 Thread Experiments:

Approach	Result	Tradeoff
Plain Map	<code>fatal error: concurrent map writes</code>	Unsafe
Mutex	Safe, but slow	Serializes all access
RWMutex	Better for read-heavy	More complex
<code>sync.Map</code>	Built-in safety	Optimized for specific patterns

Connection to distributed systems:

Goroutines racing on a shared map = the same challenge as distributed nodes disagreeing on state, just scaled across machines. Lamport's logical clocks address exactly this ordering problem.

Step II: Self-Healing & Key Readings

Resilience Test (Assignment 6):

- Killed an ECS task during 500-user load test
- ALB detected failure, ECS replaced task in $\approx$ 1 minute
- **Zero client-visible errors** — system self-healed

Readings That Shaped My Thinking:

- *Distributed Systems for Fun and Profit* — replication both causes and solves problems
- **CAP Theorem** — partitions will happen; choose consistency or availability
- **8 Fallacies of Distributed Computing** — easy to violate implicitly
- **2PC** — coordinator crash = participants stuck $\rightarrow$ motivates Paxos/Raft
- **FLP Impossibility** — consensus is provably hard, not just unsolved

Step III: The Problem

Goal: Demonstrate a service crash and recover using the **Circuit Breaker** pattern (Sam Newman, *Building Microservices*).

Setup: Go Albums API with simulated degradation:

- **Requests 1–30:** Normal — low latency responses
- **Requests 31–50:** Degraded — 1–3 second random delays
- **Requests 51+:** Crashed — HTTP 503 errors

Architecture (Broken):

Client → Albums Service (port 8080)

Step III: Crash Metrics (No Fix)

Metric	Value
Total Requests	70
Successful	50 (71.4%)
Failed	20 (28.6%)
Avg Response	683.8 ms

- 28.6% failure rate
- No fallback mechanism
- Client sees raw 503 errors

```
=== RESULTS SUMMARY ===
```

```
Total Requests: 70
```

```
Successful:      50 (71.4%)
```

```
Failed:          20 (28.6%)
```

```
Avg Response:    683.804407ms
```

```
Results saved to results_no_fix.json
```

Step III: The Fix — Circuit Breaker Pattern

Architecture (Fixed):

Client → Circuit Breaker Proxy (9090) → Albums Service (8080)

Three States:

- **CLOSED** — requests flow normally, failures counted
- **OPEN** — requests blocked, cached fallback data returned instantly
- **HALF_OPEN** — test requests sent to check if backend recovered

Configuration:

- 3 consecutive failures → OPEN
- 5 second timeout → HALF_OPEN
- 2 successes in HALF_OPEN → CLOSED

Step III: Circuit Breaker State Diagram

`CLOSED --[failure >= 3]--> OPEN`

`OPEN --[timeout 5s]--> HALF_OPEN`

`HALF_OPEN --[2 successes]--> CLOSED`

`HALF_OPEN --[failure]--> OPEN`

Fallback Strategy: When circuit is OPEN, proxy returns *cached album data* — client never sees an error. Trades **consistency** (stale data) for **availability** (no errors).

Step III: Fixed Metrics (With Circuit Breaker)

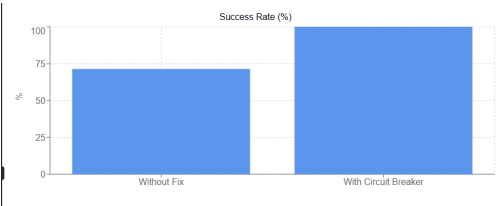
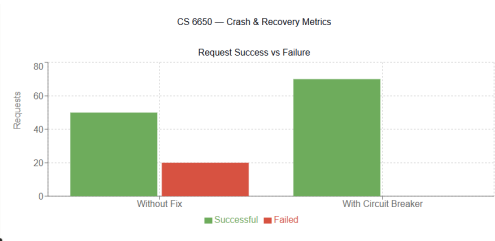
Metric	Value
Total Requests	70
Successful	70 (100%)
Failed	0 (0%)
Avg Response	832.3 ms

- **100% success rate!**
- Zero client-visible errors
- Fallback served instantly when circuit OPEN

```
=== RESULTS SUMMARY (WITH CIRCUIT BREAKER) ===  
Total Requests: 70  
Successful:    70 (100.0%)  
Failed:        0 (0.0%)  
Avg Response:  832.286244ms  
  
Results saved to results_with_fix.json
```

Step III: Before vs After

Metric	Without Fix	With CB	Change
Success Rate	71.4%	100%	+28.6%
Failed Requests	20	0	-20
Avg Response	683.8 ms	832.3 ms	—
Availability	Partial	Full	✓



Connects to Assignment 6: Same principle as ALB + ECS self-healing, but now

Key Takeaways

- 1 **Measure first** — metrics (CloudWatch, load tests) drive scaling decisions
- 2 **Design for failure** — systems *will* crash; circuit breakers, health checks, and auto-recovery are essential
- 3 **Concurrency is the root challenge** — from goroutines to distributed nodes
- 4 **CAP tradeoffs are everywhere** — circuit breaker chooses availability over consistency
- 5 **Infrastructure-as-Code** — Terraform + Docker makes deployment repeatable and scalable

The course shifted my thinking from “make it work” to “make it scale and survive.”

Thank You!

Questions?

Eroniction Presley
CS 6650 — Building Scalable Distributed Systems